NoSQL databases · Big Data Management and Processing

# Apache CouchDB 3.5.2

Replication conflicts, partitioned queries and document-level atomicity, demonstrated on a shop-order database

Daniel · Csaba

Recorded presentation · 2026

PART 1 · INTRODUCTION

# Where CouchDB comes from

**2005**
Damien Katz, a former Lotus Notes developer at IBM, starts CouchDB

**Feb 2008**
Enters the Apache Incubator under the Apache License

**Late 2008**
Graduates to an Apache top-level project

**Jul 2010**
First stable release, 1.0

**2016**
2.0 adds native clustering, from Cloudant's BigCouch

**Today**
3.5.2, the version used in this project

The goal from day one: reliable data on many unreliable machines. The name stands for **Cluster Of Unreliable Commodity Hardware**.

Sources: Wikipedia, "Apache CouchDB"; QCon SF 2010 speaker profile of Damien Katz

# Core features we use

### Documents over HTTP

JSON documents; every operation is a plain HTTP request. The Fauxton web UI is built in.

### Revisions instead of locks

Every write creates a new _rev. An update must name the current revision (MVCC).

### Multi-master replication

Any instance replicates to any other, continuously. Conflicts are detected and kept.

### Mango queries and partitions

JSON selectors backed by indexes; partitioned databases group related documents.

Source: Apache CouchDB documentation

PART 1 · INTRODUCTION

# Why it fits a multi-store order system

Our example: **5 stores × 20 orders**. Each order is one JSON document, and branches may edit while offline, then sync.

STRENGTH

## Conflicts are detected, never lost

Concurrent edits on two instances are both kept and flagged as a conflict, instead of one silently overwriting the other.

LIMITATION

## One document = one transaction

An order and its stock count cannot change together atomically, and merging conflicts is the application's job.

Both points are demonstrated in Part 5

# Project structure and how to run it

```
couchdb-project/
  run.py                    the one file you start
  run_windows.bat           the same, for Windows
  docker-compose.yml        instances A and B
  .env.example              credential template
  scripts/                  setup, seed, reset, A–D
    lib/                    HTTP client, data, merge rule
  tests/                    offline tests (pytest)
  data/                     sample data
    {} orders.json          the 100 orders
  evidence/                 results of every run
  slides/                   this presentation
  README.md                 setup and commands
  DATA_DICTIONARY.md        fields, databases, indexes
  VERIFICATION.md           what ran, on what
  MANIFEST.sha256           file checksums
```

## Built with

- **CouchDB** 3.5.2
- **Docker** 29.9.2
- **Compose** v5.6.1
- **Python** 3.14.8, stdlib only
- **pytest** 9.2.0
- **Fedora 44** Intel Core i7
- **Windows 11** WSL 2

## Try it yourself

```
$ python run.py          # everything, A to D
$ python run.py A        # one experiment
$ python run.py reset    # fresh sample data
$ python run.py test     # offline tests
$ python run.py wipe     # remove it all
```

Needs Docker and Python 3.9 or newer. Nothing else to install.

Every experiment saves its output under evidence/

PART 2 · INSTALLATION AND SETUP

# Two independent single-node instances

Fedora 44 host · Docker 29.9.2 · Compose v5.6.1 · one command: python run.py

### couch-a · instance A

127.0.0.1:5984
volume couch-a-data
admin_a · own credentials

rep-a-to-b →

← rep-b-to-a

continuous jobs,
experiment B only

### couch-b · instance B

127.0.0.1:5985
volume couch-b-data
admin_b · own credentials

**3.5.2**

image pinned by digest sha256:5fc59611...

**1 node**

in cluster_nodes on A and on B: not one cluster

**~110 MiB**

memory per container (limit 512 MiB)

**14 / 14**

setup checks passed: readiness, system DBs, write/read

Evidence: evidence/00_setup/ · ports bound to 127.0.0.1 only

# One order is one document

```
{
  "_id": "store01:order001",
  "_rev": "1-2d37e26a…",
  "type": "order",
  "tenant": "store01",
  "status": "open",
  "customer": {"name": "István Szabó", …},
  "items": [{"sku": "SKU-COF-01", "qty": 3,
             "unit_price": 18.9}],
  "total": 56.7, "currency": "EUR",
  "created_at": "2026-09-21T15:36:00Z",
  "delivery": {"city": "Budapest", …}
}
```

### _id = store : order

A natural key. The prefix is the partition key and always equals tenant.

### _rev = generation-hash

New on every write; an update must quote the current one.

### Items embedded, stock separate

An order is written as one unit. Stock lives in its own documents (see Part 5D).

### Indexes follow the queries

[tenant, status] for one store’s open orders; [tenant, status, created_at] for the sorted query.

Data: 100 orders, 5 stores, generated with a fixed seed (data/orders.json)

PART 4 · BASIC OPERATIONS

# CRUD: every write is a new revision

| Operation | Request on instance A | Response | Verified by |
|---|---|---|---|
| Create | PUT /orders/store01:order900 | 201 · rev 1-623a… | read-back, same _rev |
| Read | GET /orders/store01:order900 | 200 · status open | content matches |
| Update | PUT with _rev 1-623a…, status paid | 201 · rev 2-7826… | read-back: status paid |
| Delete | DELETE …order900?rev=2-7826… | 200 · rev 3-11ad… | GET → 404 deleted |

Delete writes a **tombstone** revision, so a replica learns about the deletion. An update that quotes an old _rev is refused with **HTTP 409** (Part 5D).

# Mango: an indexed query, not a scan

“Open orders of store01 over 100 EUR, newest first”

```
POST /orders/_find
{
  "selector": {
    "tenant": "store01",
    "status": "open",
    "total": {"$gt": 100}
  },
  "sort": [{"tenant": "desc"},
           {"status": "desc"},
           {"created_at": "desc"}]
}
```

**`tenant-status-created`**

the index _explain says the planner chose

**10 → 6**

documents examined → returned, out of 100 orders

**`same 6 ids, same order`**

as a client-side filter of every document

PART 5B · REPLICATION

# Two one-way jobs give two-way sync

**Instance A**
`couch-a / orders`

`rep-a-to-b · job stored in A's _replicator →`

`← rep-b-to-a · job stored in B's _replicator`

**Instance B**
`couch-b / orders`

| ≤ 1 s | ≤ 0.5 s | ≤ 0.5 s | 100 = 100 |
|---|---|---|---|
| both jobs "running", no errors | new order from A readable on B | new order from B readable on A | documents on A and B after catch-up |

Job URLs use the service names **couch-a** and **couch-b**: the replicator runs inside a container, where localhost means that container.

Times from the unattended run, polled every 0.5 s on one laptop

PART 5B · REPLICATION

# A controlled conflict and its merge

### 1 · COMMON START

store01:order-shared at revision **1-845c…** on A and B.

Both jobs stopped by deleting their job documents.

### 2 · DIVERGENT EDITS

**A:** status → paid
**B:** street → Kárász utca 12.

Both accepted locally with HTTP 201. No 409.

### 3 · SYNC RESUMES

New jobs, fresh ids. Both report **_conflicts** and pick the **same winner**.

The loser is still readable with ?rev=.

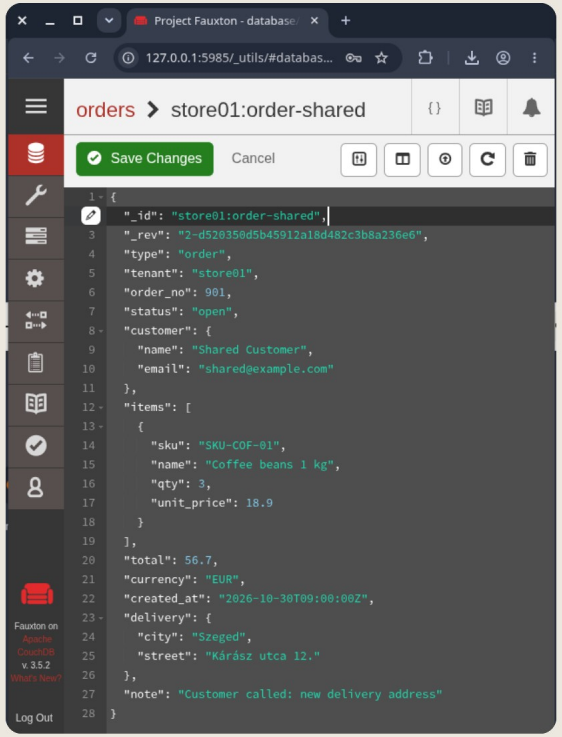
### 4 · RESOLVED

Per-field merge saved on the winner: **3-8fc0…**, paid and new street.

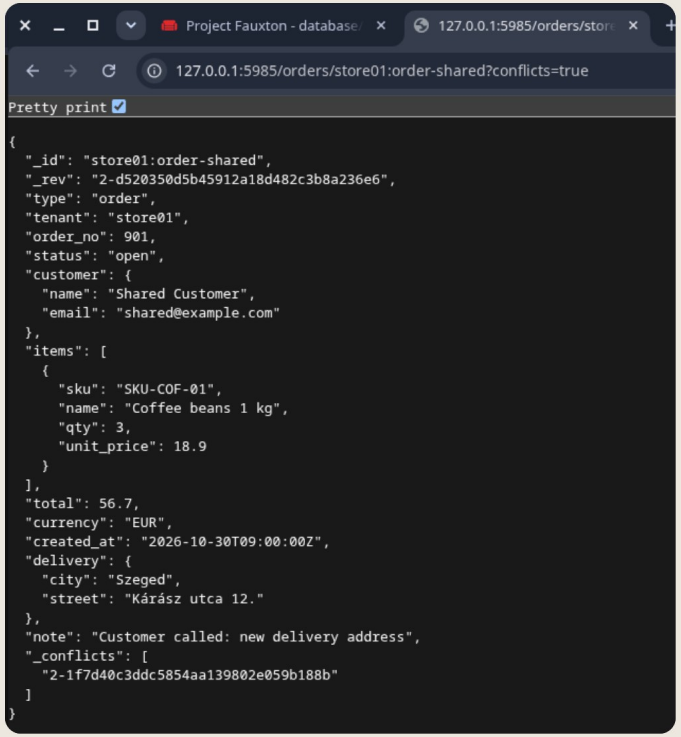
Loser deleted with its own _rev. No live conflicts on A or B.

CouchDB chooses the winner deterministically, identically on every replica, but it **never merges data**. The merge rule is ours.

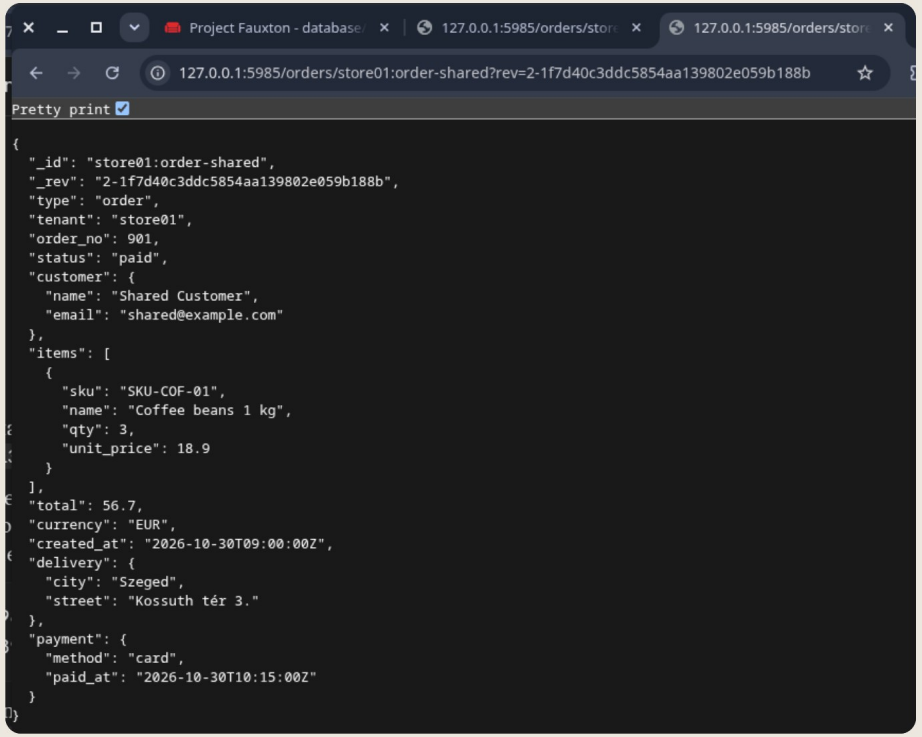
# The conflict, seen from instance B




Fauxton shows only the winner, **2-d520…**




**?conflicts=true** reveals a second branch, 2-1f7d…




Fetched by its id, the loser turns out to be A's payment.

Screenshots from python run.py B, taken before the merge step

# Partitioned vs global queries

**NORMAL DATABASE**

`POST /orders_global/_find`

global index tenant-status

**10 ids**

=

**PARTITIONED DATABASE**

`POST /orders_partitioned/_partition/store01/_find`

partitioned index tenant-status

**10 ids**

Checked with limit 100 and over every bookmark page of 3: complete, identical id sets, data unchanged.

### Needs a global view

Open orders of all stores on the partitioned DB: no usable index, a warning, 101 documents examined.

### Not claimed

Physical distribution across servers or a speed-up: one node, n=1, q=2 for both databases.

PART 5D · TRANSACTIONS

# Transactions stop at one document

### D1 · STALE UPDATE

Writer 1 updates rev 1 → rev 2 (on_hand 8). Writer 2 still holds rev 1 and tries again.

## HTTP 409 conflict

The stored value stays 8: no lost update.

### D2 · ONE _bulk_docs REQUEST

bulk-a with its current rev, bulk-b with a stale rev:

```
HTTP 201 Created
[{"ok": true, "id": "bulk-a", …},
 {"id": "bulk-b",
  "error": "conflict"}]
```

Read-back: bulk-a new, bulk-b unchanged.

An overall 201 is not success: **check every item**. A bulk request is a batch, not a transaction.

PART 5D · WHAT IT MEANS FOR THE SHOP

# Order saved.
# Stock not decremented.

“Create order + decrement stock” in one _bulk_docs request, with a stale stock revision, half-succeeded. Invariants across documents need retries or compensation, or one business event per document.

PART 6 · CONCLUSION

# What our results show

USE CASE

Offline-first order sync between shop branches or mobile tills.

STRENGTH · SEEN IN 5B

Concurrent edits on A and B were detected, kept and resolved identically on both: no silent data loss.

LIMITATION · SEEN IN 5D

Atomicity ends at one document: a bulk request half-succeeded and left order and stock inconsistent.

CouchDB fits data that is naturally one document per business event and must survive being edited in several places.

# Sources

## Apache CouchDB documentation (3.x)

- Single node setup; Installation via Docker
- Introduction to replication; Replicator database
- Replication and conflict model
- Partitioned databases
- Mango: /{db}/_find and indexes
- Bulk documents: transaction semantics
- Document API: revisions, deletion

`docs.couchdb.org/en/stable`

## History, licences, tools

- Wikipedia, “Apache CouchDB”: en.wikipedia.org/wiki/Apache_CouchDB
- QCon San Francisco 2010, speaker profile of Damien Katz
- Apache CouchDB downloads and Apache License 2.0: couchdb.apache.org
- Docker: Docker Desktop WSL 2 backend; Docker Desktop licence
- Course handout: NoSQL databases group project

Project repository: scripts, data generator, README and evidence for every experiment.